

HelixCluster

HelixCluster

Un sistema operativo distribuido para cómputo AI —desde GPUs en centros de datos hasta dispositivos portátiles en el edge, bajo un único plano de control.

Resumen

Helix Cluster OS es un sistema operativo distribuido de próxima generación que orquesta el cómputo en nodos heterogéneos — desde GPUs en centros de datos hasta SBCs en el edge y dispositivos portátiles—, unificando la programación de HPC, la orquestación de contenedores, la inferencia AI/ML, la operación federada de múltiples clústeres y sesiones multiusuario seguras bajo un único plano de control.

Descripción breve

Un clúster de cómputo distribuido basado en Go / compartición de GPU. Unifica la programación de HPC (un planificador de dos niveles basado en el modelo Omega), la orquestación de contenedores, el enrutamiento de inferencia AI, la federación y sesiones multiusuario seguras en nodos heterogéneos, coordinados mediante chismes SWIM y consenso Raft, con cifrado postcuántico de extremo a extremo.

Descripción detallada

Helix Cluster OS orquesta cargas de trabajo de cómputo en hardware radicalmente heterogéneo —GPUs en centros de datos, computadoras de placa única en el edge, incluso dispositivos portátiles— bajo un único plano de control, tratando un rack de A100s y un puñado de SBCs como una única malla direccionable en lugar de una docena de islas incompatibles. Se trata de un espacio de trabajo Go (un monorepo más submódulos de git) que implementa una pila de siete capas, desde el sustrato de hardware

L0 hasta la federación y observabilidad en L7, coordinado por catorce microservicios en el plano de control. La membresía de nodos se rastrea mediante chismes SWIM y descubrimiento, lo que permite que la malla se autorrepare a medida que los nodos se unen o abandonan; el estado fuertemente consistente se gestiona mediante consenso Raft, organizado en grupos Raft por fragmento con lecturas locales en el titular del arrendamiento para mayor velocidad y cercado STONITH para garantizar que un nodo particionado no pueda corromper el estado compartido. La ubicación de cargas de trabajo pasa por un planificador de dos niveles basado en el modelo Omega —conurrencia optimista, coincidencia ClassAd, programación en grupo, preempción por multiplicador de valor y ubicación basada en restricciones—, pero va más allá de lo que jamás hizo un planificador HPC clásico: enrutamiento consciente de la huella de carbono y el costo/TCO, escalado automático hacia la nube en ráfagas y adaptadores de mercado (Akash, io.net, RunPod, AWS Spot, Chutes) que permiten que un trabajo se desborde hacia capacidad alquilada cuando se agota el suministro local.

Los usuarios finales no ven directamente ninguna de estas complejidades; interactúan a través de un modelo de sesión limpio (asignaciones de cómputo), una terminal interactiva WebSocket/PTY, una ruta interna de inferencia AI y lecturas de utilización de recursos. La seguridad es una capa de primera clase, no un añadido: identidad SPIFFE, atestación de dispositivos (desafío/respuesta, prueba de trabajo GPU, sellado), una puerta de control de exportaciones KYC y un transporte cifrado de extremo a extremo postcuántico basado en un intercambio de claves híbrido X25519 + ML-KEM-768 con protección de registros AEAD y rechazo de repeticiones, diseñado para que el tráfico capturado hoy siga siendo confidencial incluso frente a un adversario cuántico del mañana. La corrección no se afirma, se *demuestra*: pruebas de simulación determinista (ejecuciones con semillas al estilo FoundationDB, inyección de fallos, simulación de red, reproducción byte a byte y un verificador de linealizabilidad Porcupine) reproducen fallos distribuidos bajo demanda, y las pruebas de mutación emparejadas obligatorias demuestran que las pruebas de protección realmente funcionan. La arquitectura y la documentación se mantienen fieles a la realidad mediante lintens mecánicos que hacen fallar la compilación en el momento en que la realidad y la documentación se desvían.

Por qué lo creamos

Para ejecutar cargas de trabajo AI y HPC en niveles de hardware radicalmente distintos sin tener que ensamblar planificadores, orquestadores y pilas de inferencia por separado —y hacerlo con la garantía de ingeniería de que cada funcionalidad implementada demuestra *comportamiento real del usuario final* (nunca pruebas

en verde sobre stubs) y de que cada capacidad específica del sistema operativo emplea un recurso nativo real por plataforma (sin simulaciones exclusivas de Linux). El problema que motiva el proyecto, citado en el modelo de gobernanza del repositorio, es el modo de fallo “las pruebas pasan, pero la funcionalidad no funciona en la práctica”, que el proyecto está explícitamente diseñado para eliminar.

Por qué es un cambio de paradigma

Consolida cinco elementos que normalmente son cinco pilas independientes —planificación de HPC, orquestación de contenedores, inferencia AI, federación multiclúster y sesiones multiusuario seguras— en un único plano de control que abarca desde GPUs en centros de datos hasta dispositivos de borde portátiles. Y lo logra con un rigor presupuestario reservado habitualmente para infraestructuras especializadas: corrección de nivel de métodos formales (especificaciones TLA+, simulación determinista, verificación de linealizabilidad) y transporte confidencial postcuántico, el tipo de garantías que la mayoría de los orquestadores ni siquiera intentan. Además de la diferenciación técnica, la colocación consciente de costes y emisiones, junto con la capacidad de escalar a mercados en la nube, lo convierten también en una *palanca económica*: el planificador puede buscar automáticamente capacidad más barata, ecológica o sobrante, de modo que una misma carga de trabajo cueste menos y genere menos emisiones sin que nadie tenga que reescribir el trabajo.

Qué lo hace innovador

- **Pruebas de simulación determinista (DST)** — un simulador con semilla, totalmente reproducible, que inyecta fallos, desviaciones de reloj y particiones de red, las reproduce byte a byte y ejecuta el resultado a través de un verificador de linealizabilidad Porcupine, de modo que un *Heisenbug* detectado una vez pueda reproducirse a demanda para siempre.
- **Planificador de dos niveles modelo Omega** — colocación con concurrencia optimista, emparejamiento mediante ClassAd, planificación por grupos y preempción basada en multiplicadores de valor, un diseño de estado compartido que permite a múltiples planificadores operar sobre un mismo clúster sin cuellos de botella centrales.
- **Cableado de inferencia confidencial con cifrado E2EE postcuántico** — un intercambio de claves híbrido X25519 + ML-KEM-768 con vinculación de pares de claves de respuesta por solicitud y AEAD con rechazo de repeticiones (los primitivos criptográficos son reales y probados; el recorrido

confidencial multinodo completo sigue explícitamente en fase PLANIFICADA/bloqueada).

- **Confianza basada en atestación** — los nodos deben *demonstrar* qué son: identidad SPIFFE, prueba de trabajo GPU, sellado de dispositivo, una puerta de control de exportaciones KYC y generación de documentación conforme al Reglamento AI de la UE, de modo que la confianza se gana mediante evidencia, no se asume por posición en la red.
- **Orquestación consciente de costes y emisiones** — modelado de TCO, colocación con conciencia de huella de carbono, escalado a la nube, reserva de conmutación por error N+K y adaptadores para mercados en la nube, convirtiendo el precio y las emisiones en entradas de primera clase para la planificación, en lugar de consideraciones secundarias.
- **Consenso Multi-Raft** — grupos Raft por fragmento con lecturas locales en el titular del arrendamiento para consistencia de baja latencia, respaldados por cercado STONITH (IPMI/EC2/Azure/SBD), de modo que un nodo bloqueado se elimina de forma decisiva, sin riesgo de corromper el estado.
- **Límites mecánicos contra la deriva** — archlint falla la compilación en el instante en que un componente documentado apunta a una ruta de paquete inexistente, y un motor de cadena de documentación mantiene la coherencia byte a byte entre Markdown/HTML/PDF/DOCX, de modo que la documentación no pueda mentir silenciosamente sobre el código.

Mayores desafíos técnicos y cómo los resolvimos

- **“PASS-bluff” (pruebas que pasan en funciones no operativas).** El modo de fallo que todo el proyecto busca erradicar: un conjunto de pruebas en verde sobre *stubs*. Se solucionó con pruebas de mutación emparejadas obligatorias —cada elemento de trabajo incluye una prueba de guardia nominada que debe *fallar* ante una mutación independiente del código antes de marcarse como completado, de modo que una prueba que pasa demuestre ejercer un comportamiento real en lugar de un *mock*—.
- **Paridad multiplataforma (sin *mocks* exclusivos de Linux).** Se resolvió con una interfaz compartida dividida por etiquetas de compilación en facilidades genuinas por sistema operativo —Linux con *cgroup* / *proc* / WireGuard del kernel, macOS con *sysctl* / *vm_stat* / IOKit / *wireguard-go*— y luego verificada con un oráculo independiente por sistema operativo, para que cada plataforma reporte su estado nativo real en lugar de una ficción basada en Linux.
- **Corrección distribuida bajo fallos.** Se solucionó con pruebas de simulación determinista y un verificador de

linealizabilidad que genera y reproduce particiones, caídas y desincronización de relojes, respaldado por especificaciones formales TLA+ que definen los invariantes de consenso y programación antes de escribir una sola línea de código.

- **Desviación en documentación y arquitectura.** Se resolvió con *archlint*, que falla la compilación ante cualquier paquete documentado pero inexistente, y una verificación *docs_chain* sin escapatoria —la desviación se convierte en un error de compilación, no en una página wiki obsoleta—.
- **Delimitación honesta del trabajo incompleto.** El recorrido de inferencia multinodo confidencial está deliberadamente protegido por un *ticket* y etiquetado como “aún no validado de extremo a extremo” en lugar de presentarse como entregado —la misma disciplina se aplica a lo que *no* está hecho que a lo que *sí*—.

Pila tecnológica

- **Go (go.mod: 1.25 / toolchain 1.26.4)** — el lenguaje del plano de control en un espacio de trabajo de ~30 módulos; elegido por su concurrencia económica con *goroutines* y binarios estáticos que se despliegan de manera idéntica desde el centro de datos hasta el *edge*—.
- **Zig (0.14+) + C/C++** — utilizado exactamente donde el *runtime* de Go se interpone: en primitivas de sistema de bajo nivel y núcleos GPU que requieren control determinista y libre de asignaciones sobre el *hardware*—.
- **gRPC + Protocol Buffers** — cada API entre subsistemas (*api/v1/*) es un contrato tipado y versionado, de modo que los catorce microservicios evolucionan sin romperse entre sí ni crear formatos de intercambio manuales—.
- **Raft (etcd-raft) + gossip SWIM** — una división intencional: Raft gestiona el estado que *debe* ser fuertemente consistente, mientras que el *gossip* SWIM maneja la membresía y el descubrimiento a escala, donde el consenso sería demasiado pesado—.
- **PostgreSQL 16, Redis 7, etcd v3.5, SQLite** — el almacén adecuado para cada tarea: PostgreSQL para estado relacional duradero, Redis para caché en caliente, etcd para coordinación y SQLite embebido para el registro local de elementos de trabajo HXC—.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — tres *backbones* de mensajería para tres tipos de tráfico: NATS/*JetStream* para eventos internos rápidos, Kafka para flujos duraderos de alto rendimiento y RabbitMQ para semántica clásica de *broker*—.
- **Malla WireGuard + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard para una malla ligera nodo a nodo, envuelta en un apretón de manos híbrido poscuántico y registros AEAD, de modo que el

transporte sea confidencial tanto frente a atacantes clásicos como cuánticos—.

- **SPIFFE + JWT (HS256) + RBAC basado en ámbitos + OPA** — identidad y autorización en capas: SPIFFE para identidad de carga de trabajo, JWT para *tokens*, RBAC basado en ámbitos para acceso grueso y OPA para expresar políticas granulares como código—.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, trazado W3C** — métricas, paneles y trazas distribuidas con propagación de contexto W3C, para seguir una solicitud a través de servicios y capas de *hardware*—.
- **HashiCorp Vault 1.16** — secretos y material criptográfico mantenidos fuera del código y la configuración, emitidos bajo auditoría—.
- **Docker Compose, Kubernetes (*kustomize*, *securityContext* endurecido), Helm** — Compose para el despliegue local y Kubernetes/Helm con contextos de seguridad endurecidos para despliegues reales, una misma definición promovida en todos los entornos—.
- **React + TypeScript + Vite (Node 20+)** — una interfaz web rápida y con tipado seguro para sesiones, terminales y utilización de *pools*—.
- **TLA+** — especificación formal de los invariantes de consenso y programación, de modo que las propiedades más difíciles de probar se demuestran a nivel de diseño antes de la implementación—.

Notas sobre estado y honestidad

- **Estado: en desarrollo.** La versión es temprana (0.1.0-dev). Varias funciones avanzadas —inferencia confidencial multinodo de ida y vuelta, liquidación en el mercado y población de programación basada en atestaciones— están explícitamente etiquetadas como PLANIFICADAS / con infraestructura restringida en el repositorio y **no** se presentan como completamente funcionales. Las cifras de cobertura son autoinformadas.
- **Licencia: por determinar.** No está claramente declarada; los enlaces HelixCluster/HelixCluster y las URL de helixcluster.io del gráfico Helm son marcadores de posición no verificados que no coinciden con los repositorios reales.
- Los proyectos de la pila LLM incluidos (LLMOrchestrator, LLMProvider, LLMsVerifier) son submódulos desacoplados, no servidores de modelos alojados dentro del clúster.

Nivel de prioridad: Helix-principal (clúster LLM-infraestructura —el sustrato de cómputo que puede alojar cargas de trabajo de inferencia y cómputo). Tiene menor prioridad que HelixTrack.